PANDUAN SETUP CI/CD DAN NASKAH VIDEO CI/CD

Aplikasi Keluh Kesah — Kelompok Koboy

Dokumen ini berisi dua bagian. Bagian A adalah langkah setup satu kali agar pipeline GitHub Actions bisa men-deploy aplikasi ke laptop salah satu anggota. Bagian B adalah naskah video penjelasan CI/CD untuk tiga pembicara. Kerjakan Bagian A sampai pipeline hijau sebelum merekam Bagian B.

Peran	Nama	Tugas
Pemilik laptop deploy	(pilih satu anggota)	Memasang runner, menjalankan run.cmd, menyalakan laptop saat demo
Admin repository	Wan Saka Nasa	Mengisi secrets, membuat environment, push file CI/CD
Pembicara video	Wan Saka, Joni, Percaya	Sesuai pembagian di Bagian B

A. Setup CI/CD (sekali saja)

Urutannya penting: secrets dan runner disiapkan dulu, baru file CI/CD di-push. Kalau file di-push lebih dulu, pipeline langsung berjalan dan job Delivery gagal karena secret Docker Hub belum ada.

A1. Buat access token Docker Hub

1. Login ke hub.docker.com dengan akun narasandrone.
2. Klik foto profil → Account settings → Personal access tokens → Generate new token.
3. Isi Access token description: `github-actions-kiboy`, Expiration: sesuai kebutuhan (misal 90 hari), Access permissions: **Read & Write**.
4. Klik Generate, lalu salin token yang muncul. Token hanya ditampilkan sekali; simpan sementara di Notepad.

A2. Isi GitHub Secrets

Buka https://github.com/WanSaka8/kiboy → Settings → Secrets and variables → Actions → New repository secret. Buat lima secret berikut (nama harus persis sama, huruf besar semua):

Name	Secret (isi)
DOCKERHUB_USERNAME	narasandrone

Name	Secret (isi)
DOCKERHUB_TOKEN	token dari langkah A1
MYSQL_PASSWORD	password user database, misal koboy123456
MYSQL_ROOT_PASSWORD	password root MySQL, misal koboy1234567
JWT_SECRET	string acak panjang, misal hasil dari https://randomkeygen.com

Tips: bila laptop deploy sudah pernah menjalankan aplikasi, pakai password yang sama dengan file `.env` di laptop itu. Kalau berbeda, jalankan `docker compose down -v` sekali di folder project laptop sebelum deploy pertama (data lama terhapus dan dibuat ulang dari `init.sql`).

A3. Buat environment production dan amankan runner

1. Settings → Environments → New environment → Name: `production` → Configure environment. Tidak perlu mengubah apa pun lagi, cukup simpan.
2. Settings → Actions → General → bagian **Fork pull request workflows from outside collaborators** → pilih **Require approval for all external contributors** → Save. Ini mencegah orang luar menjalankan kode di laptop kalian lewat pull request.
3. Masih di halaman yang sama, pastikan **Actions permissions** diatur ke Allow all actions and reusable workflows.

A4. Pasang self-hosted runner di laptop deploy

Dikerjakan oleh pemilik laptop. Syarat: Windows 10/11 64-bit, Docker Desktop terpasang dan berjalan, serta Git for Windows terpasang (menyediakan Git Bash).

1. Buka Git Bash, cek `docker compose version` dan `bash --version`. Keduanya harus menampilkan versi.
2. Di GitHub: Settings → Actions → Runners → **New self-hosted runner** → pilih **Windows**, Architecture **x64**.
3. Buka PowerShell, buat folder runner: `mkdir C:\actions-runner; cd C:\actions-runner`.
4. Salin dan jalankan perintah **Download** dan **Extract** yang ditampilkan GitHub di halaman tersebut (perintahnya berisi versi runner terbaru).
5. Jalankan perintah **Configure** dari halaman GitHub, tambahkan label: `.\config.cmd --url https://github.com/WanSaka8/kiboy --token <TOKEN-DARI-HALAMAN> --labels keluhkesah`.

6. Saat ditanya: runner group → Enter; name → misal `laptop-koboy`; additional labels → Enter (label keluhkesah sudah dari parameter); work folder → Enter; **run as service → ketik N**.
7. Jalankan `.\run.cmd`. Tunggu sampai muncul tulisan `Listening for Jobs`. Jendela PowerShell ini harus tetap terbuka selama pipeline dipakai.
8. Cek di GitHub: Settings → Actions → Runners. Runner `laptop-koboy` tampil dengan status **Idle** dan label `self-hosted`, `Windows`, `X64`, `keluhkesah`.

Runner sengaja tidak dijalankan sebagai Windows service. Service berjalan sebagai akun sistem yang tidak bisa mengakses Docker Desktop milik pengguna, sehingga job deploy akan gagal. Setiap kali laptop dinyalakan ulang, buka Docker Desktop lalu jalankan lagi `C:\actions-runner\run.cmd`.

A5. Push file CI/CD ke GitHub

1. Ekstrak `kiboy_siap_push.zip` ke folder repository `kiboy` di laptop (yang terhubung ke GitHub), pilih **Replace** untuk semua file. Isinya: semua file repo ditambah `.github/workflows/ci-cd.yml`, `scripts/smoke-test.sh`, `.gitattributes`, `.env.example`, `docker-compose.yml` dan `nginx.conf` yang sudah diperbaiki, serta README baru.
2. Di Git Bash, dari folder `kiboy`: `git status` untuk melihat file yang berubah.
3. `git add .` lalu `git commit -m "Tambah pipeline CI/CD GitHub Actions"`.
4. `git push origin main`. Pipeline langsung berjalan.

A6. Cek hasil pipeline pertama

Tempat	Yang harus terlihat
https://github.com/WanSaka8/kiboy/actions	Run "CI/CD Keluh Kesah" dengan tiga job: 1. CI - Build & Test, 2. Delivery - Push ke Docker Hub, 3. Deploy - Laptop (self-hosted), semuanya centang hijau. Waktu total sekitar 4–7 menit.
Jendela run.cmd di laptop	Baris `Running job: 3. Deploy - Laptop (self-hosted)` lalu `Job ... completed with result: Succeeded`.
https://hub.docker.com/r/narasandrone/izinkompe/tags	Tag `backend`, `frontend`, serta `backend-<sha>` dan `frontend-<sha>` dengan waktu push beberapa menit lalu.
Git Bash di laptop: docker ps	Container keluhkesah-db, keluhkesah-backend, keluhkesah-frontend (healthy) dan keluhkesah-adminer berjalan.

Tempat	Yang harus terlihat
http://localhost:8080 di laptop	Aplikasi terbuka dan bisa login demo / demo1234.
Halaman utama repo, kolom kanan	Bagian Deployments menampilkan production dan badge CI/CD di README berwarna hijau (passing).

Ambil screenshot keempat bukti pertama untuk subbab 4.10 laporan.

A7. Mengatasi masalah yang sering muncul

Gejala	Penyebab	Solusi
Job 3 lama "Waiting for a runner to pick up this job"	run.cmd tidak berjalan, atau label bukan keluhkesah	Jalankan C:\actions-runner\ run.cmd; cek label di Settings → Actions → Runners
Job 2 gagal: unauthorized: incorrect username or password	Secret DOCKERHUB_USERNAME/TOKEN salah atau token hanya Read	Buat token baru dengan izin Read & Write, perbarui secret
Job 3 gagal: error during connect ... dockerDesktopLinuxEngine	Docker Desktop di laptop belum dibuka	Buka Docker Desktop, tunggu Engine running, klik Re-run failed jobs
Job 3: container keluhkesah-backend is unhealthy, log Access denied for user keluh	Password secret berbeda dengan password saat volume dbdata pertama dibuat	Sekali saja: docker compose down -v di laptop, lalu Re-run jobs
$'\r': command not found saat smoke test	smoke-test.sh tersimpan dengan line ending Windows (CRLF)	Pastikan .gitattributes ikut ter-push, lalu git add --renormalize . dan push ulang
Bind for 0.0.0.0:8080 failed: port is already allocated	Port 8080 dipakai aplikasi lain (XAMPP, Apache, dll.)	Matikan aplikasi tersebut, lalu Re-run jobs
Smoke test gagal di langkah login	Akun demo sudah dihapus dari database laptop	docker compose down -v sekali agar init.sql membuat ulang akun demo

B. Naskah Video Implementasi CI/CD

Durasi sekitar 10 menit, satu kali rekam dengan OBS seperti video container. Inti video adalah demonstrasi langsung: satu perubahan kode di-push, lalu dalam beberapa menit muncul sendiri di aplikasi yang berjalan di laptop tanpa perintah Docker manual.

B1. Persiapan sebelum rekam

- Bagian A sudah selesai dan minimal satu run pipeline sudah hijau.
- Docker Desktop terbuka, `run.cmd` berjalan dengan status `Listening for Jobs`, dan aplikasi sudah terbuka di http://localhost:8080.
- VS Code membuka folder `kiboy` dengan zoom 150%: tab `.github/workflows/ci-cd.yml`, `scripts/smoke-test.sh`, dan `frontend/index.html` (baris 1560).
- Tab browser: halaman repo GitHub, tab Actions, Settings → Secrets and variables → Actions, Settings → Actions → Runners, Docker Hub tags, dan http://localhost:8080.
- Perubahan untuk demo belum di-commit. Rencananya: ubah `<p>Tempat berbagi cerita dan keluh kesah</p>` menjadi `<p>Tempat berbagi cerita dan keluh kesah – dideploy otomatis dengan CI/CD</p>`.
- Waktu pipeline sekitar 4–7 menit. Kalau lebih lama, pakai tombol Pause OBS sambil menunggu.

B2. Pembagian peran

Bagian	Pembicara	Waktu	Isi
1	Wan Saka Nasa	00:00–03:30	Pembukaan, konsep pipeline, file workflow: pemicu dan job CI
2	Joni Fernando S	03:30–06:30	Job Delivery dan Deploy, smoke test, secrets, self-hosted runner
3	Percaya Mendrofa	06:30–10:00	Demo langsung push → pipeline → Docker Hub → aplikasi berubah, penutup

B3. Bagian 1 – Wan Saka Nasa (00:00–03:30)

Waktu	Di layar	Ucapan
00:00	Halaman repo GitHub, badge CI/CD hijau di README	"Selamat pagi, Pak. Kami Kelompok Koboy: saya Wan Saka Nasa, bersama Joni Fernando dan Percaya Mendrofa. Di video ini kami menjelaskan implementasi CI/CD untuk aplikasi Keluh Kesah, mulai dari kode di-push ke GitHub sampai aplikasi ter-deploy otomatis di laptop."
00:30	Gambar 4.1 dari laporan (diagram pipeline)	"Pipeline kami memakai GitHub Actions dan terdiri dari tiga job. Job 1, CI, membangun image dan menguji keempat container. Job 2, Delivery, mengirim image yang lulus uji ke Docker Hub. Job 3, Deploy, berjalan di laptop kami yang dipasangi self-hosted runner, lalu menarik image baru dan menjalankannya."
01:15	VS Code: ci-cd.yml baris 5–19	"Semua pipeline ada di satu file, .github/workflows/ci-cd.yml. Pipeline berjalan saat ada push ke main, saat ada pull request, atau dijalankan manual. Untuk pull request, hanya job CI yang jalan, jadi kode diuji dulu sebelum digabung."
02:00	ci-cd.yml baris 22–52	"Job 1 berjalan di server GitHub ubuntu-latest. File .env dibuat dari .env.example, lalu docker compose build membangun image backend dan frontend. docker compose up --wait menunggu sampai db, backend, dan frontend berstatus healthy. Setelah itu skrip smoke test menguji aplikasinya. Kalau gagal, log semua container ditampilkan."
03:00	ci-cd.yml baris 54–72	"Kalau lulus dan ini push ke main, image yang sudah diuji disimpan sebagai artifact untuk job berikutnya, lalu semua container dimatikan." Serah terima: "Selanjutnya Joni menjelaskan job Delivery dan Deploy."

B4. Bagian 2 – Joni Fernando S (03:30–06:30)

Waktu	Di layar	Ucapan
03:30	ci-cd.yml baris 74–102	"Job 2 hanya jalan kalau job 1 lulus. Image tidak dibangun ulang; artifact dari job 1 dimuat dengan docker load, jadi image yang dikirim sama persis dengan yang diuji. Setelah login ke Docker Hub, setiap image di-push dengan dua tag: backend dan frontend untuk versi terbaru, plus tag dengan hash commit untuk riwayat versi."

Waktu	Di layar	Ucapan
04:15	ci-cd.yml baris 104–137	"Job 3 memakai runs-on self-hosted dengan label keluhkesah, artinya job ini dikirim ke laptop kami. File .env ditulis dari GitHub Secrets, jadi password database dan JWT secret tidak pernah ada di repository."
04:50	Settings → Secrets and variables → Actions	"Ini lima secret yang kami simpan: username dan token Docker Hub, dua password MySQL, dan JWT_SECRET. Nilainya tidak bisa dilihat lagi, dan di log otomatis disamarkan."
05:10	ci-cd.yml baris 139–157	"Di laptop, docker compose pull menarik image terbaru, lalu up -d --no-build --wait mengganti container backend dan frontend. Container database tidak dibuat ulang, jadi data tetap aman di volume. Lalu smoke test yang sama dijalankan di laptop."
05:40	scripts/smoke-test.sh	"Smoke test menguji enam hal: health API dan database, halaman utama, endpoint tanpa token harus ditolak 401, login akun demo, cek identitas token, dan data cerita terbaca dari MySQL. Kalau satu saja gagal, skrip keluar dengan kode 1 dan pipeline berhenti."
06:10	Settings → Actions → Runners, lalu jendela run.cmd	"Ini runner di laptop kami, statusnya Idle dengan label keluhkesah, dan di jendela ini runner sedang menunggu job." Serah terima: "Sekarang Percaya akan mendemokan pipeline-nya secara langsung."

B5. Bagian 3 – Percaya Mendrofa (06:30–10:00)

Waktu	Di layar	Ucapan
06:30	Browser http://localhost:8080, subjudul lama	"Ini aplikasi yang sedang berjalan di laptop. Perhatikan subjudulnya: Tempat berbagi cerita dan keluh kesah."
06:45	VS Code index.html baris 1560, ubah teks, simpan	"Saya ubah subjudulnya menjadi: dideploy otomatis dengan CI/CD."
07:00	Git Bash: `git add .`, `git commit -m "Ubah subjudul untuk demo CI/CD"`, `git push origin main`	"Lalu saya commit dan push ke main. Setelah ini saya tidak menjalankan perintah Docker apa pun."

Waktu	Di layar	Ucapan
07:20	Tab Actions → klik run terbaru	"Pipeline langsung berjalan. Job 1 sedang build dan menguji container." Buka langkah Smoke test aplikasi: "Ini hasil smoke test: semua baris OK."
08:20	Job 2 selesai, buka Docker Hub tags	"Job Delivery selesai. Di Docker Hub muncul tag frontend dan frontend dengan hash commit yang baru saja di-push."
08:50	Job 3 berjalan; tampilkan jendela run.cmd lalu log job	"Job Deploy diambil oleh runner di laptop ini. Image ditarik, container diganti, dan smoke test di laptop lulus."
09:20	Refresh http://localhost:8080	"Setelah di-refresh, subjudulnya sudah berubah. Cerita yang sudah ada juga masih ada, karena database tidak disentuh."
09:35	Halaman repo: Deployments production, badge hijau	"Riwayat deploy tercatat di environment production. Kalau versi baru bermasalah, cukup git revert lalu push, dan pipeline men-deploy versi sebelumnya. Sekian video CI/CD dari Kelompok Koboy, terima kasih, Pak."

B6. Setelah rekam

1. Edit di Clipchamp: potong waktu tunggu pipeline yang terlalu lama, beri judul "Kelompok Koboy – Implementasi CI/CD".
2. Upload ke OneDrive Mikroskil, Share → People in Universitas Mikroskil atau Anyone with the link (Can view), cek di jendela Incognito.
3. Kirim link video CI/CD dan link video container yang baru, beserta screenshot bukti pipeline, supaya dimasukkan ke subbab 4.10 dan bab 6 laporan.